



Project Report

Volatility Voyage

Summer 2025

Mentors:

Anurag Thakur
Arihant Satpathy
Kethan Challa
Shivam Tomar

CONTENTS

1	WEEK 0: Introduction to Finance	3
1.1	Session Overview	3
1.2	What Our Mentors Taught Us	3
1.2.1	Introduction to EDA and its importance in Financial world . . .	3
1.2.2	Trading Strategies	3
1.2.3	Understanding Volatility	4
1.2.4	Backtesting, Signal generation and Metric Computation	4
1.3	Assignment Context and Computational Steps	4
1.3.1	Code Highlights	4
1.4	Key Takeaways	5
1.5	Conclusions and Reflections	5
2	Week 1: Volatility-Aware Trading Strategies	6
2.1	Session Overview	6
2.2	Strategy Building Progression	6
2.2.1	Strategy 1 – EMA Crossover	6
2.2.2	Strategy 2 – EMA + Dynamic Band	6
2.2.3	Strategy 3 – ATR Breakout	6
2.3	Risk Metrics Deep Dive	7
2.3.1	Value at Risk (VaR)	7
2.3.2	Conditional Value at Risk (CVaR)	7
2.4	Portfolio Sizing via Volatility	7
2.5	Backtesting Engine: Theory	7
2.5.1	Purpose	7
2.5.2	Core Components	7
2.6	Evaluation Metrics	8
2.7	Final Plots	8
2.7.1	Portfolio Value Curve	8
2.7.2	Buy/Sell Signal Markers	9
2.7.3	Drawdown Plot	9
2.7.4	Capital Allocation Dynamics (India VIX)	10
2.8	Code Snippet	11
2.9	Insights & Learnings	11
3	Week 2: Risk-Managed Trading Strategies	12
3.1	Session Overview	12
3.2	Strategy Framework	12
3.3	Risk Management Mechanisms	12
3.4	Backtesting Metrics	13
3.5	Performance Results	13
3.6	Key Insights	13

4	Week 3: Volatility Forecasting and Portfolio Optimization	14
4.1	Session Overview	14
4.2	Portfolio Theory	14
4.2.1	Risk and Return	14
4.2.2	Mean-Variance Optimization	14
4.2.3	Portfolio Variance and Diversification	15
4.3	Systematic and Unsystematic Risk	15
4.4	Efficient Frontier	15
4.5	Tangency Portfolio and Capital Market Line	15
4.5.1	Sharpe Ratio	15
4.5.2	CAPM and Beta	15
4.6	Hands-on Application: Indian Stocks	16
4.7	Volatility Forecasting Methods	18
4.7.1	Rolling Mean	18
4.7.2	Exponentially Weighted Moving Average (EWMA)	18
4.7.3	AR(1) Model	18
4.8	Investor Risk Profiling and Allocation	19
4.9	Portfolio Construction and Backtesting	20
4.9.1	Capital Allocation	20
4.9.2	Backtesting Framework	20
4.10	Conclusion and Insights	21
5	Week 4: Extended Volatility Models and Hedging	21
5.1	ARMA Model	21
5.2	ARIMA Model	22
5.3	GARCH Model	23
5.4	EGARCH Model	24
6	Final Hedging Strategy (Capital-Neutral Long–Short)	24
6.1	Final Results	26
6.2	Final Metrics	26

1 WEEK 0: INTRODUCTION TO FINANCE

1.1 Session Overview

In Week 1, we embarked on the foundational journey of understanding stock behavior through data analysis and basic volatility modeling. Our mentors emphasized the importance of developing intuition for market behavior using real-world stock data before diving into advanced modeling or forecasting.

The session revolved around hands-on exploratory data analysis (EDA) and the introduction of two pivotal concepts in finance: trading strategies and volatility via standard deviation. Mentors also introduced us to backtesting and metrics for portfolio evaluation, along with relevant code.

1.2 What Our Mentors Taught Us

1.2.1 Introduction to EDA and its importance in Financial world

The mentors kicked off the session by introducing the concept of Exploratory Data Analysis (EDA) in a financial context. EDA is not just about graphs – it's about storytelling with data. We explored historical stock prices using descriptive statistics (mean, median, standard deviation, min, max) and visual tools like line charts and boxplots to understand price behavior, detect outliers, and spot trends.

Key EDA metrics included:

- Daily closing price trend
- Price range and outliers
- Price volatility through time
- Distribution of returns

This stage helped ground our understanding in data before we added any modeling layers.

1.2.2 Trading Strategies

We were then introduced to the four main types of trading strategies - trend following, mean reversion, breakout and volatility based strategies and some of the widely used indicators. We then saw one of the most basic yet widely-used techniques in action —Moving Average Convergence and Divergence.

Two types of moving averages were implemented:

- Simple Moving Average (SMA): The unweighted mean of the previous n prices.
- Exponential Moving Average (EMA): Similar to SMA but with more weight on recent prices, making it more responsive to new information.

Along with the 9 day-signal line in a sample notebook.

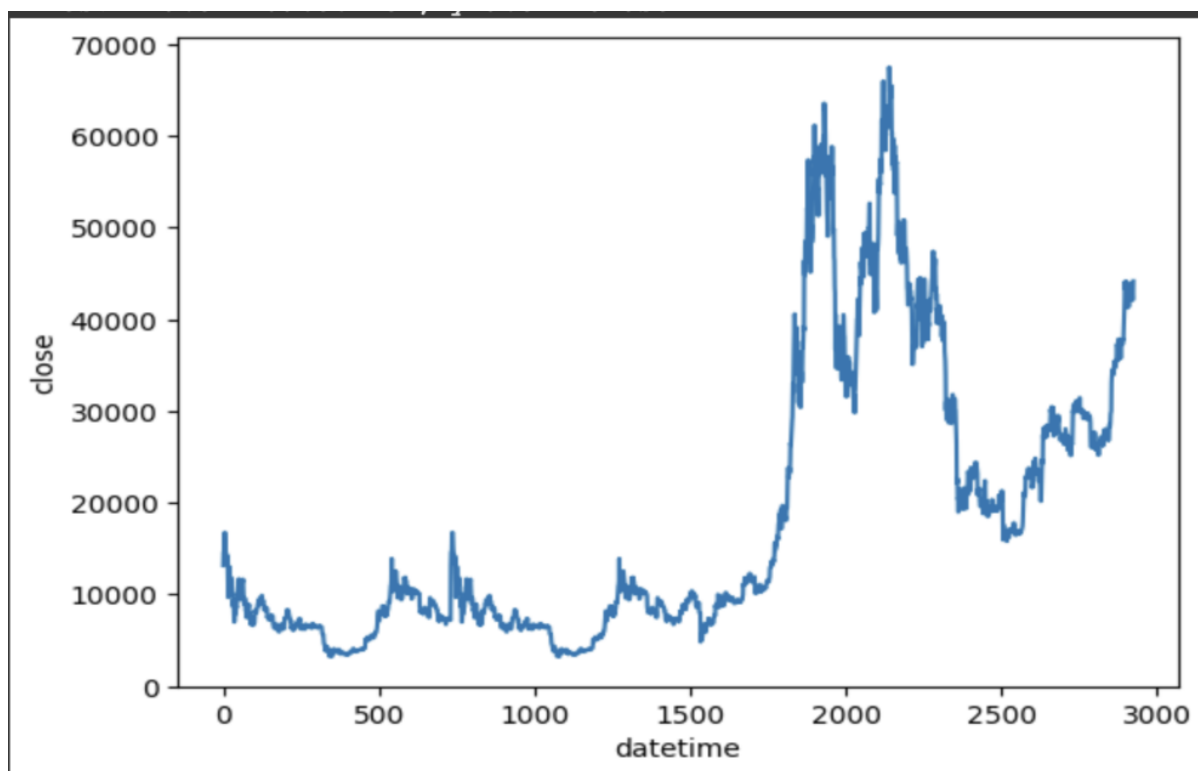


Figure 1: Stock Price Movement

1.2.3 Understanding Volatility

The mentors explained volatility through a simple yet powerful analogy—market mood swings. Some stocks stay calm and predictable, while others are prone to extreme highs and lows.

To quantify this, we used **standard deviation of price**—a direct statistical representation of volatility.

We also learned about VIX (Volatility Index, measures expected short-term volatility in the US stock market).

1.2.4 Backtesting, Signal generation and Metric Computation

Code for signal generation using MACD strategy and metric computation was thoroughly explained and mentees were shown its implementation on real world data (Apple stock). We as mentees were advised to learn how to code our own backtesting engine from scratch.

1.3 Assignment Context and Computational Steps

1.3.1 Code Highlights

The assignment had two major components:

- Data Preparation and Analysis
 - i. Performed EDA using `.describe()`, histograms, and time-series plots

```
def macdline(data):
    data['EMA26']=data['Close'].ewm(span=26,adjust=False).mean()
    data['EMA12']=data['Close'].ewm(span=12,adjust=False).mean()
    macd=[]
    for i in range(len(data)):
        x=data['EMA12'].iloc[i]-data['EMA26'].iloc[i]
        macd.append(x)

    data['MACD Line']=macd
    return data
```

Figure 2: Moving Average Convergence and Divergence

```
def signal(data):
    sig = []
    #sig.append(0)
    for i in range(0, len(data)):
        if data['MACD Line'].iloc[i] > data['Signal Line'].iloc[i] and data['MACD Line'].iloc[i - 1] < data['Signal Line'].iloc[i - 1]:
            sig.append(1)
        elif data['MACD Line'].iloc[i] < data['Signal Line'].iloc[i] and data['MACD Line'].iloc[i - 1] > data['Signal Line'].iloc[i - 1]:
            sig.append(-1)
        else:
            sig.append(0)
    data['Signal'] = sig
```

Figure 3: Signal Generator

- ii. Added SMA, EMA20, and EMA50 to the dataset using pandas' rolling() and ewm() methods
- Volatility Analysis
 - i. Computed 6-month rolling standard deviation
 - ii. Isolated and printed the most volatile 6-month period

1.4 Key Takeaways

- EDA is the first step in analysing raw data - patterns and anomalies reveal themselves best when seen.
- SMA and EMA are fundamental tools for trend detection and market signal filtering.
- Standard deviation is a robust starting point for understanding risk and market dynamics (volatility measurement).
- Identifying the “most volatile” period helps frame investment strategy, stop-loss levels, and capital allocation decisions.

1.5 Conclusions and Reflections

Assignment 1 was our first step into market analysis using real data and code. The simplicity of tools like SMA, EMA, and standard deviation belied their utility. While basic, these indicators are foundational in any risk-aware trading or investment strategy.

The hands-on coding component helped bridge theory with data intuition and python programming skills. The first session set the foundation for deeper dive in the world of trading strategies and portfolio analysis.

2 WEEK 1: VOLATILITY-AWARE TRADING STRATEGIES

2.1 Session Overview

This week focused on evolving from basic moving average-based strategies to volatility-aware trading systems. The use of technical indicators like Exponential Moving Average (EMA) and Average True Range (ATR) was augmented with risk management metrics such as Value at Risk (VaR) and Conditional Value at Risk (CVaR). A major theme was the integration of India VIX to dynamically adjust capital allocation, simulating real-time responses to market uncertainty.

2.2 Strategy Building Progression

2.2.1 Strategy 1 – EMA Crossover

This is a trend-following strategy where trades are initiated based on the crossover of price with a moving average:

- **Buy Signal:** When Close price crosses above the EMA.
- **Sell Signal:** When Close price crosses below the EMA.

The EMA is calculated using:

$$EMA_t = \alpha \cdot Price_t + (1 - \alpha) \cdot EMA_{t-1}$$

where $\alpha = \frac{2}{n+1}$ and n is the window size.

2.2.2 Strategy 2 – EMA + Dynamic Band

Builds upon Strategy 1 by introducing a volatility band defined as:

$$\text{Buy Band} = EMA + (Close - Open), \quad \text{Sell Band} = EMA - (Close - Open)$$

Trades are executed when price crosses above the Buy Band or below the Sell Band.

2.2.3 Strategy 3 – ATR Breakout

This strategy utilizes the Average True Range (ATR) to adjust the band dynamically:

$$\text{True Range (TR)} = \max(\text{High} - \text{Low}, |\text{High} - \text{Close}_{\text{prev}}|, |\text{Low} - \text{Close}_{\text{prev}}|)$$

$$ATR = EMA \text{ of TR}$$

$$\text{Buy Band} = EMA + ATR, \quad \text{Sell Band} = EMA - ATR$$

2.3 Risk Metrics Deep Dive

2.3.1 Value at Risk (VaR)

VaR estimates the potential loss in a portfolio over a given time period at a certain confidence level (e.g., 95%). Mathematically:

$$\text{VaR}_{95\%} = \text{Percentile}_{5\%}(\text{Daily Returns})$$

2.3.2 Conditional Value at Risk (CVaR)

CVaR estimates the expected loss given that the loss is beyond the VaR threshold:

$$\text{CVaR}_{95\%} = \mathbb{E}[\text{Loss} \mid \text{Loss} > \text{VaR}]$$

This captures tail risk better than VaR.

2.4 Portfolio Sizing via Volatility

To manage exposure dynamically, India VIX and Close price volatility were used:

$$\text{Threshold}_1 = \mu + 0.5\sigma, \quad \text{Threshold}_2 = \mu + 1.5\sigma$$

where μ is the mean and σ is the standard deviation of percentage changes in VIX.

Capital Allocation Rules:

- $\text{VIX} < \text{Threshold}_1$: Allocate 100%
- $\text{Threshold}_1 \leq \text{VIX} < \text{Threshold}_2$: Allocate 75%
- $\text{VIX} \geq \text{Threshold}_2$: Allocate 50%

2.5 Backtesting Engine: Theory

2.5.1 Purpose

Backtesting simulates historical performance of a strategy using past data. It helps in estimating expected returns, drawdowns, and risks before live deployment.

2.5.2 Core Components

- **Signal Generator:** Identifies buy/sell moments.
- **Trade Execution Engine:** Simulates trades based on signals and capital.
- **Capital Allocation Logic:** Adjusts quantity traded based on VIX thresholds.
- **Metrics Collector:** Records performance metrics per trade and per day.

2.6 Evaluation Metrics

1. Net Profit (%):

$$\text{Net Profit} = \frac{\text{Final Portfolio Value} - \text{Initial Capital}}{\text{Initial Capital}} \times 100$$

2. Gross Profit (%):

$$\text{Gross Profit} = \frac{\sum \text{Positive Trade Profits}}{\text{Initial Capital}} \times 100$$

3. Sharpe Ratio:

$$\text{Sharpe} = \frac{\bar{R} - R_f}{\sigma_R}$$

where $\bar{R}$ is average return, R_f is risk-free rate, σ_R is return std deviation.

4. Drawdown (%): Peak-to-trough decline in equity curve.

5. Dip (%):

$$\text{Dip} = \frac{\text{Entry Price} - \text{Min Interim Price}}{\text{Entry Price}} \times 100$$

6. Holding Period: Average and max duration a trade was held.

7. Number of Trades: Total, wins, losses.

2.7 Final Plots

2.7.1 Portfolio Value Curve

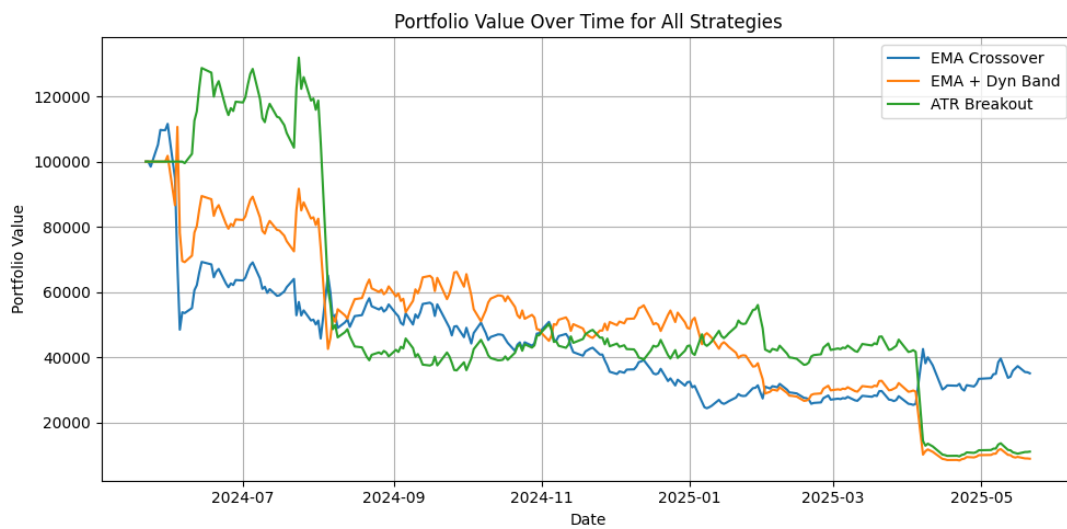


Figure 4: Portfolio Value Over Time for All Strategies

2.7.2 Buy/Sell Signal Markers

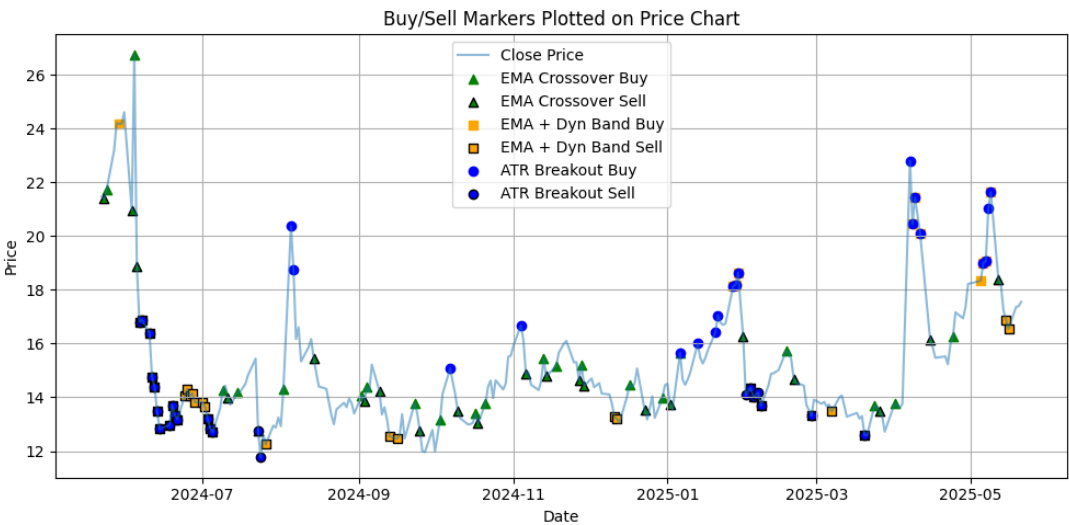


Figure 5: Buy/Sell Markers Plotted on Price Chart

2.7.3 Drawdown Plot

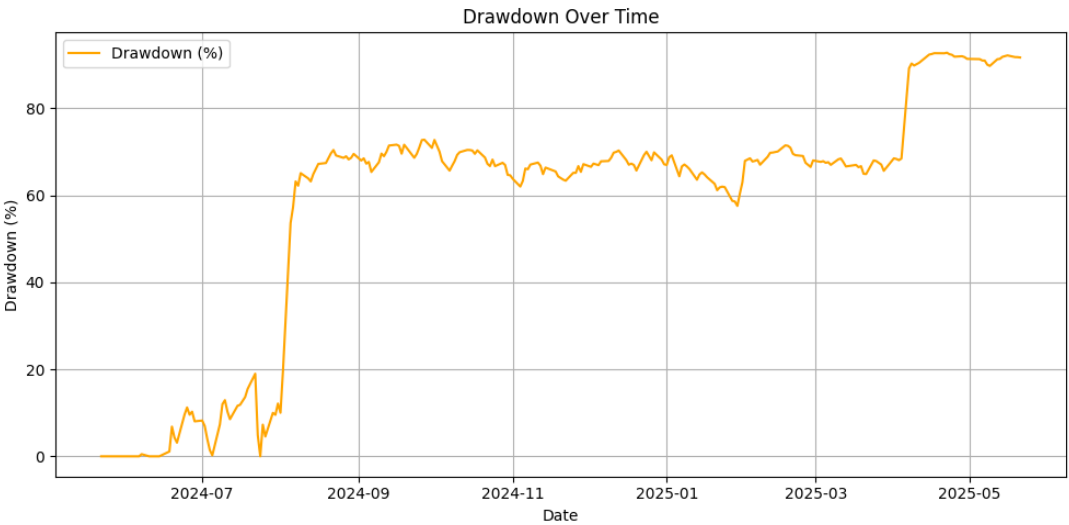


Figure 6: Drawdown Plot Showing Risk Over Time

2.7.4 Capital Allocation Dynamics (India VIX)

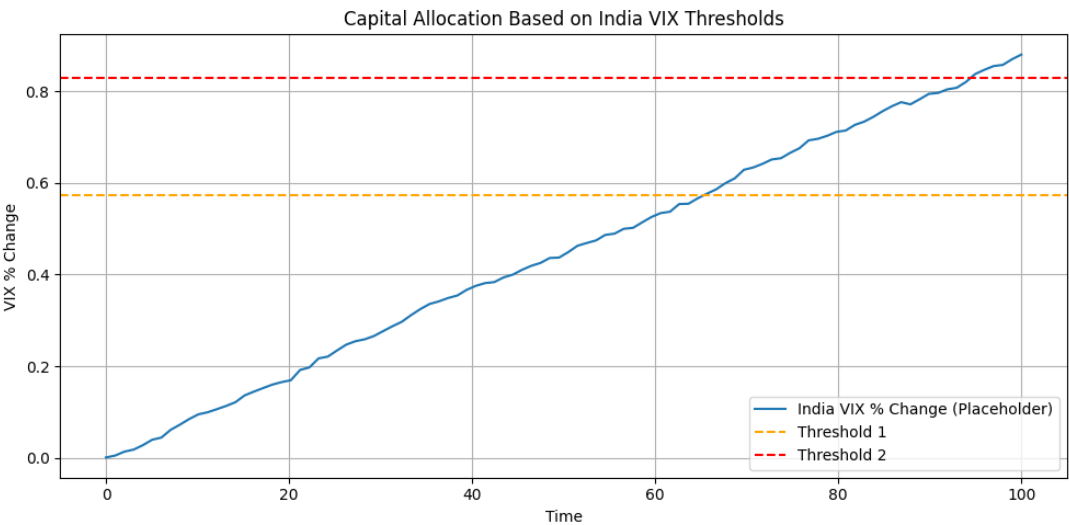


Figure 7: Capital Allocation Based on India VIX Thresholds

2.8 Code Snippet

```
def compute_signals(df):
    close = df['Close'].values
    high = df['High'].values
    low = df['Low'].values
    ema14 = pd.Series(close, index=df.index).ewm(span=14).mean().values

    # ATR calculation
    prev_close = np.concatenate([[close[0]], close[:-1]])
    tr = np.maximum.reduce([
        high - low,
        np.abs(high - prev_close),
        np.abs(low - prev_close)
    ])
    atr14 = pd.Series(tr, index=df.index).ewm(span=14).mean().values

    s1 = np.zeros(len(df))
    s2 = np.zeros(len(df))
    s3 = np.zeros(len(df))

    for i in range(1, len(df)):
        if close[i] > ema14[i] and close[i - 1] <= ema14[i - 1]:
            s1[i] = 1
        elif close[i] < ema14[i] and close[i - 1] >= ema14[i - 1]:
            s1[i] = -1

        band = high[i] - low[i]
        if close[i] > ema14[i] + band:
            s2[i] = 1
        elif close[i] < ema14[i] - band:
            s2[i] = -1

        if close[i] > ema14[i] + 0.8 * atr14[i]:
            s3[i] = 1
        elif close[i] < ema14[i] - 0.8 * atr14[i]:
            s3[i] = -1

    df['Signal1'], df['Signal2'], df['Signal3'] = s1, s2, s3
    return df
```

Figure 8: Strategy Implementation Snippet in Python

2.9 Insights & Learnings

- EMA-based strategies are effective in trending markets but fail in ranging periods.
- ATR-based bands offer better breakout filtering in volatile environments.
- Dynamic capital allocation using VIX reduces drawdowns and improves stability.
- VaR and CVaR help identify and control tail risk in the strategy.

3 WEEK 2: RISK-MANAGED TRADING STRATEGIES

3.1 Session Overview

Week 2 focused on enhancing trading strategies with risk management techniques for BTC-USD. The session emphasized moving beyond signal generation to incorporate capital protection mechanisms like stop losses, volatility-based exits, and position sizing. Key takeaways included the importance of drawdown control and risk-reward optimization in real-world trading.

3.2 Strategy Framework

A hybrid momentum-mean-reversion strategy was implemented:

- **Entry Signals:** Moving average crossover.
- **Signal Filtering:** Bollinger Band deviation to reduce false positives.
- **Volatility Filter:** ATR-based regime filtering for trade eligibility.



3.3 Risk Management Mechanisms

Five exit methods were integrated:

- Trailing Stop Loss (TSL) to lock profits.
- ATR-Based Stop Loss for dynamic exits.
- Take-Profit (TP) at fixed multiples.
- Drawdown-Based Exit for portfolio protection.

- Time-Based Exits to avoid stagnant positions.

Risk Management

```
def atr(df):
    high = list(df['High'])
    low = list(df['Low'])
    close = list(df['Close'])
    tr = [high[0] - low[0]]
    for i in range(1, len(df)):
        tr.append(max(high[i] - low[i], abs(high[i] - close[i-1]), abs(low[i] - close[i-1])))
    df['true_range'] = tr
    df['atr'] = df['true_range'].ewm(span=14, adjust=True).mean()
    del df['true_range']
    return df
df = atr(df)
buy_df = df[df['Signal'] == 1]
sell_df = df[df['Signal'] == -1]

fig=go.Figure()
fig.add_trace(go.Scatter(x=buy_df.index, y=buy_df['Close'], mode='markers', name='Buy', marker=dict(color='green', symbol='triangle-up', size=12)))
fig.add_trace(go.Scatter(x=sell_df.index, y=sell_df['Close'], mode='markers', name='Sell', marker=dict(color='red', symbol='triangle-down', size=12)))
fig.add_trace(go.Scatter(x=df.index, y=df['Close'], name='Close Price', line=dict(color='blue')))
fig.add_trace(go.Scatter(x=df.index, y=df['atr']*10, name='ATR', line=dict(color='orange'), opacity=0.5))

fig.show()
```

3.4 Backtesting Metrics

The enhanced backtesting engine tracked:

- Cumulative portfolio value and equity curves.
- Key metrics: Max Drawdown, Sharpe Ratio, Win/Loss Ratio.
- Trade statistics: Total trades, holding periods, long/short distribution.

3.5 Performance Results

Metric	Without Risk Control	With Risk Control
Net Profit (%)	High but volatile	Slightly lower but stable
Max Drawdown (%)	Very high	-46.95%
Sharpe Ratio	Unstable	2.55
Holding Period	Longer	32.44 days
Trade Consistency	Low	Improved

3.6 Key Insights

- Risk management reduced raw returns marginally but improved stability (Sharpe Ratio: 2.55).
- Trailing stops excelled in trending markets; ATR stops adapted to sideways conditions.
- Drawdowns were halved compared to the unmanaged strategy.

- Risk control optimizes reward-to-risk rather than sacrificing returns.

```
Benchmark Returns: 239.0625065918969%
Portfolio Returns: 857.7857562255859%
Total trades executed: 36
Win rate: 33.333333333333336%
Total Long Trades: 18
Total Short Trades: 18
Average Drawdown: -25.116672026177223%
Max Drawdown: -46.95002349132768%
Sharpe Ratio: 2.5533530780271874
Sortino Ratio: 3.364869207564304
Average Holding Time: 32.44444444444444 days
Maximum Holding Time: 196 days
```

4 WEEK 3: VOLATILITY FORECASTING AND PORTFOLIO OPTIMIZATION

4.1 Session Overview

Week 3 focused on Portfolio Theory and Volatility Forecasting. We covered the theoretical backbone of optimal asset allocation using mean-variance optimization and the Efficient Frontier, and implemented volatility forecasting methods to construct investor-specific portfolios. Practical exercises involved estimating risk metrics from historical data and backtesting portfolio performance.

4.2 Portfolio Theory

4.2.1 Risk and Return

Investors prefer high returns and low risk. Risk is quantified as the standard deviation of returns:

$$\sigma_p = \sqrt{\text{Var}[R_p]}$$

4.2.2 Mean-Variance Optimization

Objective is to either:

- Maximize expected return for a given level of risk.

- Minimize risk for a target return.

Efficient portfolios lie on the **Efficient Frontier**, forming the set of non-dominated portfolios in risk-return space.

4.2.3 Portfolio Variance and Diversification

Portfolio variance depends on:

- Individual asset variances
- Covariances between assets

For two assets:

$$\text{Var}[R_p] = \omega_a^2 \sigma_a^2 + \omega_b^2 \sigma_b^2 + 2\omega_a \omega_b \sigma_a \sigma_b \rho_{ab}$$

Negative correlations ($\rho_{ab} < 0$) significantly reduce risk through diversification.

4.3 Systematic and Unsystematic Risk

- **Unsystematic Risk:** Asset-specific; diversified away.
- **Systematic Risk:** Market-wide; cannot be diversified away.

4.4 Efficient Frontier

Displays optimal portfolios. Shape is influenced by correlations between assets:

- Lower correlation $\Rightarrow$ greater curvature $\Rightarrow$ better diversification.

4.5 Tangency Portfolio and Capital Market Line

Adding a risk-free asset allows us to draw the Capital Market Line (CML):

- Tangency portfolio has highest Sharpe Ratio.
- All investors combine it with the risk-free asset depending on their risk tolerance.

4.5.1 Sharpe Ratio

Risk-adjusted performance metric:

$$\text{Sharpe Ratio} = \frac{E[R_p] - R_f}{\sigma_p}$$

Higher Sharpe Ratio implies a better risk-return tradeoff.

4.5.2 CAPM and Beta

$$E[R_i] = R_f + \beta_i(E[R_m] - R_f)$$

- β_i : Sensitivity to market movements.
- Security Market Line visualizes over/under-valuation.

4.6 Hands-on Application: Indian Stocks

Three Indian stocks (e.g., Reliance, Infosys, ICICI Bank) were used to:

- Compute expected return, volatility, and covariance

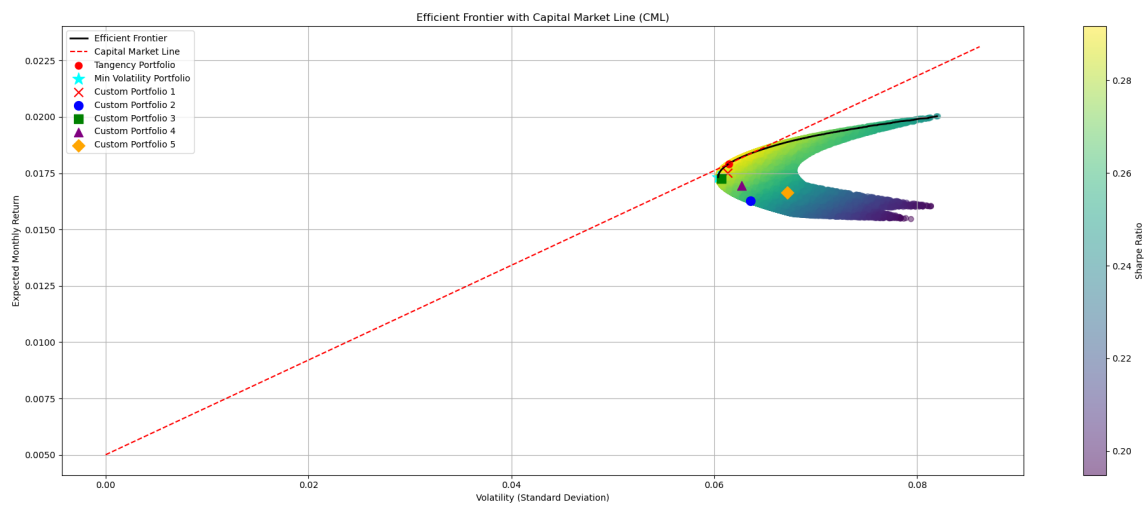
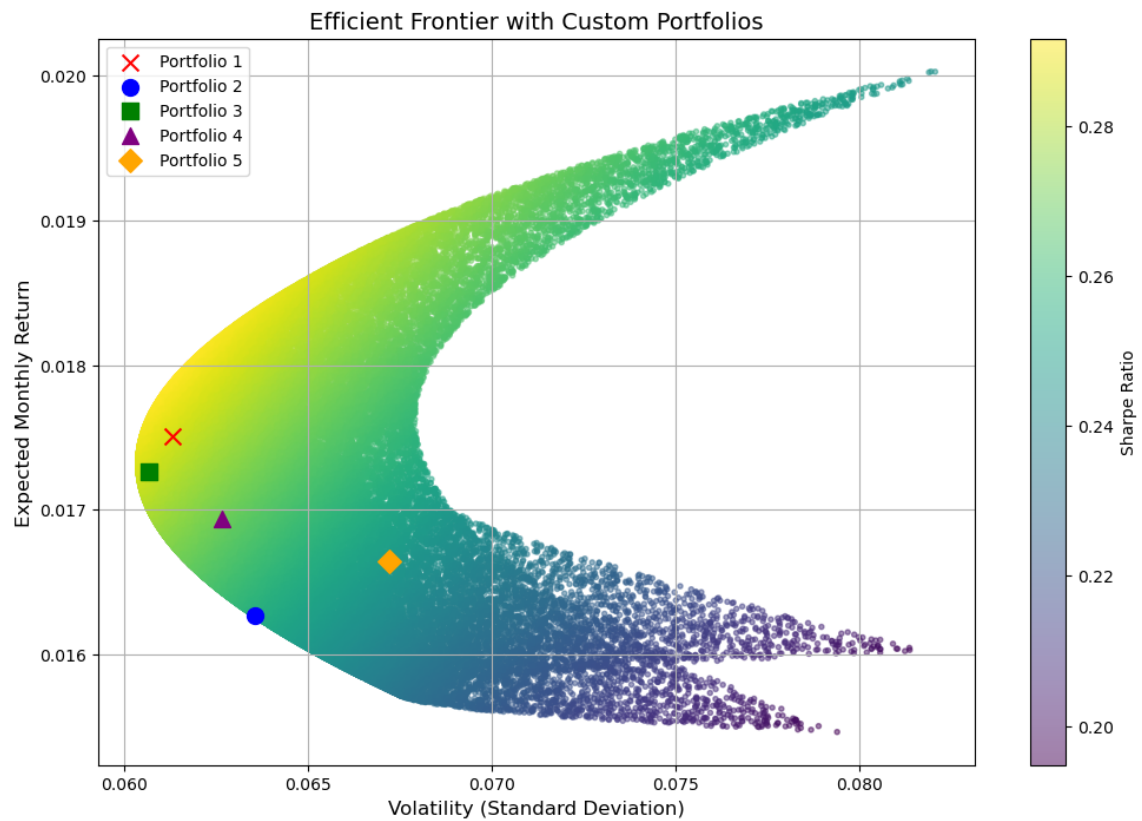
```
cov = {}
for i in tickers:
    L = []
    for j in tickers:
        I = list(returns[i])
        J = list(returns[j])
        mean_I = sum(I)/len(I)
        mean_J = sum(J)/len(J)
        l = [(I[k] - mean_I)*(J[k] - mean_J) for k in range(len(J))]
        L.append(sum(l)/len(l))
    cov[i] = L
cov_df = pd.DataFrame(cov)
cov_df.index = tickers
cov_df
```

- Simulate multiple portfolios

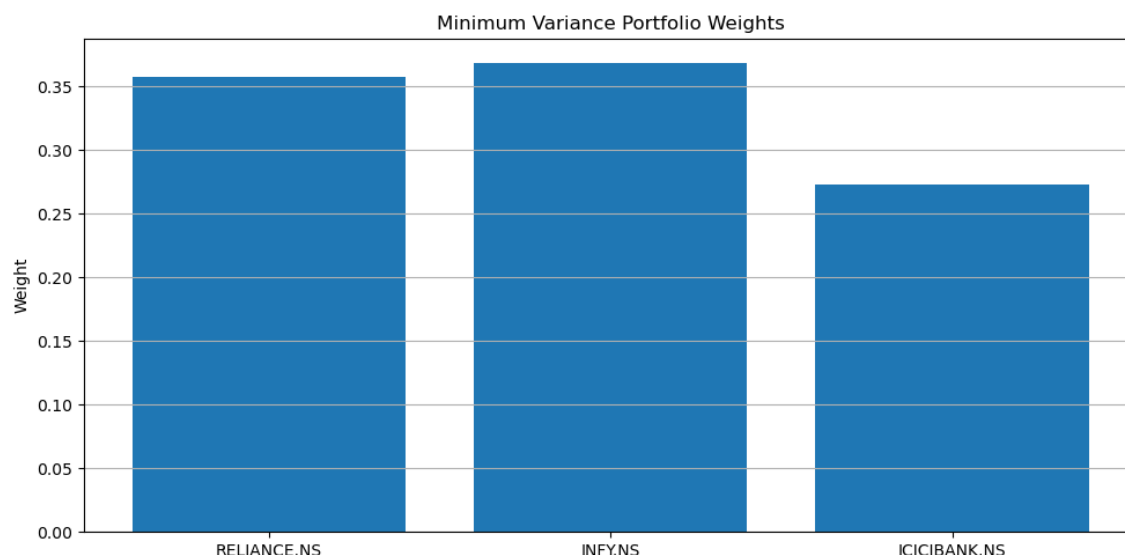
	Portfolio1	Portfolio2	Portfolio3	Portfolio4	Portfolio5
RELIANCE.NS	0.159900	0.036302	0.254332	0.377255	0.463888
INFY.NS	0.435475	0.259426	0.292386	0.250873	0.490939
ICICIBANK.NS	0.404624	0.704271	0.453282	0.371873	0.045173

- Plot Efficient Frontier and find:
 - Minimum Variance Portfolio (MVP)
 - Tangency Portfolio

It is generally seen that the overall risk is reduced when we introduce diversification. It may seem tempting to keep introducing a higher number of assets to drive down risk, but systematic risk obviously remains, and it becomes redundant to keep adding a higher number of stocks; rather, it might complicate things.



We can explore how changing the weights gives us portfolios with various risk-reward patterns, and we arrive at the Minimum Variance Portfolio and Tangency portfolio in doing so.



4.7 Volatility Forecasting Methods

4.7.1 Rolling Mean

Simple method where standard deviation is calculated over a rolling window of past returns:

$$\sigma_t = \sqrt{\frac{1}{n} \sum_{i=t-n}^{t-1} (r_i - \bar{r})^2}$$

4.7.2 Exponentially Weighted Moving Average (EWMA)

More recent observations are weighted more heavily:

$$\sigma_t^2 = (1 - \lambda)r_{t-1}^2 + \lambda\sigma_{t-1}^2$$

4.7.3 AR(1) Model

Autoregressive model using past volatility:

$$\sigma_t = \alpha + \beta\sigma_{t-1} + \epsilon_t$$

```
for ticker in tickers:
    data = yf.download(ticker, start=start_date, end=end_date, progress=False)
    # Rolling Mean Forecast
    data['returns'] = data['Close'].pct_change()
    data['rolling_vol'] = data['returns'].rolling(window=20).std()

    data['vol_forecast_rm'] = data['rolling_vol'].rolling(window=5).mean()
    # Exponentially Weighted Moving Average (EWMA)
    lambda_ = 0.94
    data['vol_squared'] = data['returns']**2
    data['ewma_vol'] = data['vol_squared'].ewm(alpha=1 - lambda_).mean() ** 0.5
    # AR(1) model on Volatility
    model = AutoReg(data['rolling_vol'].dropna(), lags=1).fit()

    true_vol = data['rolling_vol'].dropna()
    ar1_pred = model.params['const'] + model.params['rolling_vol.L1'] * true_vol.shift(1)
```

4.8 Investor Risk Profiling and Allocation

Investors are assigned a risk score $r \in [0, 1]$. Stocks are ranked by predicted volatility. Allocation is based on the investor's risk profile:

- Low-risk investors $\rightarrow$ Choose low-volatility stocks
- High-risk investors $\rightarrow$ Include high-volatility stocks

Let forecast volatility for a stock be v . The normalized volatility is given by:

$$\tilde{v} = \frac{v - v_{\min}}{v_{\max} - v_{\min}} \in [0, 1]$$

```
result_df['normalized_vol'] = (result_df['forecasted_vol'] - result_df['forecasted_vol'].min()) / \
                             (result_df['forecasted_vol'].max() - result_df['forecasted_vol'].min())
# result_df
```

We then select the three stocks whose normalized predicted volatility $\tilde{v}$ is closest to the investor's risk score i.e the absolute value of their differences is lowest r .

```
risk_aversion_score = 0.5
result_df['distance'] = abs(result_df['normalized_vol'] - risk_aversion_score)
result_df.sort_values('distance', inplace=True)
# result_df

top_stocks = result_df.head(3).copy()
top_stocks
```

	forecasted_vol	normalized_vol	distance
HDFCLIFE.NS	0.017667	0.490273	0.009727
TECHM.NS	0.018144	0.513945	0.013945
ADANIPTS.NS	0.018263	0.519834	0.019834

4.9 Portfolio Construction and Backtesting

4.9.1 Capital Allocation

- **Equal Weighting:** Equal capital for each selected stock.
- **Volatility Weighting:** Allocate more to stocks with lower volatility.

```
initial_investment = 1000000

inv_vol = 1 / top_stocks['forecasted_vol'] # we can also manage portfolio in bases of distance also
top_stocks['weight'] = inv_vol / inv_vol.sum()

# Final allocation
top_stocks['investment'] = (top_stocks['weight'] * initial_investment).round(2)
top_stocks
```

4.9.2 Backtesting Framework

Simulate portfolio over time:

- Use forecasted volatility and price data
- Rebalance portfolio periodically
- Track returns, volatility, and Sharpe Ratio

```
Benchmark Return: 24.402571818840002 %
Gross Profit: 39.09484399322464 %
Max Holding Time: 103
Average Holding Time: 39.35925925925926
Total Trades: 30
Winning Trades: 13
Losing Trades: 17
Max Drawdown: -18.053747958166444 %
Sharpe Ratio: 0.363034766120804
```



4.10 Conclusion and Insights

- Portfolio risk depends not just on individual volatilities, but on covariances.
- Diversification is powerful, especially with low-correlated assets.
- Volatility forecasting allows better alignment with investor preferences.
- Optimal portfolios lie on the Efficient Frontier or Capital Market Line.
- Risk-adjusted returns (Sharpe Ratio) are key to portfolio comparison.

5 WEEK 4: EXTENDED VOLATILITY MODELS AND HEDGING

5.1 ARMA Model

Theory

The Autoregressive Moving Average (ARMA) model is a combination of two linear time-series models:

- **Autoregressive (AR):** Models the current value as a linear combination of its previous p values.
- **Moving Average (MA):** Models the current value as a linear combination of past q error terms.

An $\text{ARMA}(p, q)$ process is defined as:

$$X_t = c + \sum_{i=1}^p \phi_i X_{t-i} + \varepsilon_t + \sum_{j=1}^q \theta_j \varepsilon_{t-j}$$

where ε_t is white noise.

When to Use

ARMA is suitable for stationary time series where both autoregressive patterns and moving average patterns are present.

Insights

- Captures short-term dependencies and moving average effects.
- Requires stationarity; non-stationary data should be differenced first.
- Parameters p and q selected via ACF/PACF or information criteria (AIC, BIC).

5.2 ARIMA Model

Theory

The Autoregressive Integrated Moving Average (ARIMA) model extends ARMA by incorporating differencing to handle non-stationary time series.

- p : Autoregressive terms.
- d : Differencing order.
- q : Moving average terms.

The ARIMA(p, d, q) process can be expressed as:

$$\Delta^d X_t = c + \sum_{i=1}^p \phi_i \Delta^d X_{t-i} + \varepsilon_t + \sum_{j=1}^q \theta_j \varepsilon_{t-j}$$

where Δ^d denotes differencing d times.

When to Use

ARIMA is suitable for non-stationary time series that become stationary after differencing.

Insights

- Generalizes ARMA for non-stationary data.
- Parameters chosen using ACF/PACF and stationarity tests.
- Over-differencing can cause loss of information.
- Useful for mean forecasting, but not direct volatility modelling.

5.3 GARCH Model

Theory

The Generalized Autoregressive Conditional Heteroskedasticity (GARCH) model captures time-varying volatility by modelling current variance as a function of past squared residuals and past variances:

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^p \alpha_i \varepsilon_{t-i}^2 + \sum_{j=1}^q \beta_j \sigma_{t-j}^2$$

This allows volatility clustering to be captured effectively.

When to Use

GARCH is suitable when volatility changes over time and exhibits persistence.

Code Snippet

```

return vol_forecast_df, avg_forecasted_vol_df

def garch_forecast_series(returns_df, tickers, window=90, p=1, q=1, dist='t', forecast_horizon=1):
    vol_forecast_dict = {}
    avg_forecasted_vol_list = []

    for ticker in tqdm(tickers):
        rolling_volatility = []
        dates = []
        ret_series = returns_df[ticker]

        for i in range(window, len(ret_series)):
            train_data = ret_series.iloc[i - window:i]

            if train_data.isna().any() or train_data.std() < 1e-6:
                continue

            try:
                model = arch_model(train_data * 100, vol='Garch', p=p, q=q, dist=dist, mean='Constant')
                result = model.fit(displ='off', show_warning=False)
                forecast = result.forecast(horizon=forecast_horizon, reindex=False)
                var = forecast.variance.values[-1, 0] / (100**2)

                vol_annual = np.sqrt(var * 252)

                if not np.isnan(vol_annual) and vol_annual < 100:
                    rolling_volatility.append(vol_annual)
                    dates.append(ret_series.index[i])
            except Exception:
                continue

        vol_series = pd.Series(rolling_volatility, index=dates)
        vol_forecast_dict[ticker] = vol_series

        avg_forecasted_vol = np.mean(rolling_volatility) if rolling_volatility else np.nan
        avg_forecasted_vol_list.append(avg_forecasted_vol)
        print(f"avg_forecasted_vol of {ticker} = {avg_forecasted_vol}")

    vol_forecast_df = pd.DataFrame(vol_forecast_dict)
    avg_forecasted_vol_df = pd.DataFrame(avg_forecasted_vol_list, index=returns_df.columns, columns=['avg_vol'])

    return vol_forecast_df, avg_forecasted_vol_df

```

Figure 9: Python Implementation of GARCH Model

Results

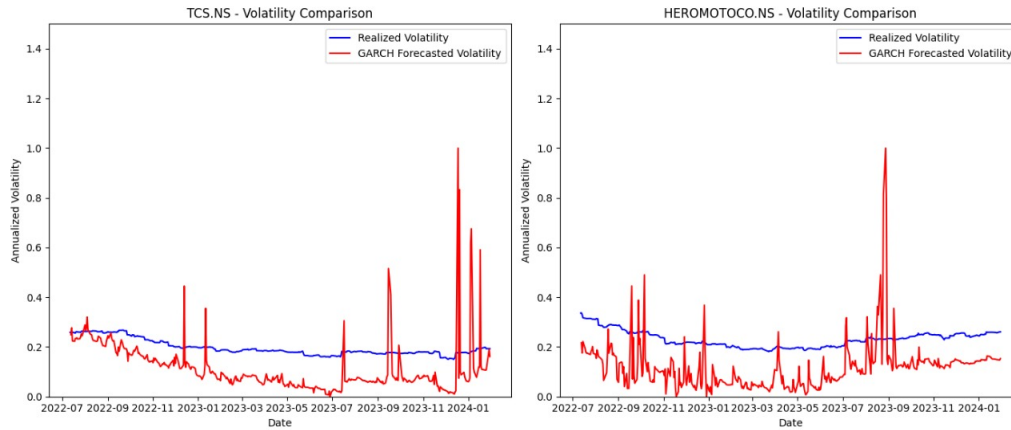


Figure 10: Volatility Forecasts using GARCH

5.4 EGARCH Model

Theory

The Exponential GARCH (EGARCH) model extends GARCH by modelling the log of conditional variance:

$$\log(\sigma_t^2) = \omega + \sum_{i=1}^p \beta_i \log(\sigma_{t-i}^2) + \sum_{j=1}^q \alpha_j \frac{\varepsilon_{t-j}}{\sigma_{t-j}} + \gamma_j \left(\left| \frac{\varepsilon_{t-j}}{\sigma_{t-j}} \right| - E \left| \frac{\varepsilon_{t-j}}{\sigma_{t-j}} \right| \right)$$

This captures asymmetric volatility effects (leverage effect).

When to Use

EGARCH is preferred when negative returns have a stronger impact on volatility than positive returns.

6 FINAL HEDGING STRATEGY (CAPITAL-NEUTRAL LONG-SHORT)

Objective

To isolate model-specific forecasting skill (pure alpha) by removing broad market risk. We hedge each long position in our top 2 EGARCH-selected stocks with a short position in its most positively correlated NIFTY 50 stock.

Methodology

1. **Stock Selection:** From our GARCH ranking (June 2022 – May 2024), pick the top two stocks with the strongest forecast performance or signal strength.

2. **Log Returns:** Calculate daily log returns for all NIFTY 50 stocks:

$$r_t = \log \left(\frac{P_t}{P_{t-1}} \right)$$

3. **Correlation Analysis:** For each selected stock:

- Compute the Pearson correlation of its log returns with all other NIFTY 50 stocks (excluding itself).
- Identify the stock with the highest positive correlation.

4. **Hedging Rule:** For each selected stock:

- Take a long position in the selected stock.
- Take an equal capital short position in its most correlated stock:

$$\text{Capital}_{\text{long}} = \text{Capital}_{\text{short}}$$

This creates a capital-neutral position and reduces systematic risk.

5. **Hedged P&L:** The daily hedged profit and loss is calculated as:

$$\text{Hedged P\&L}_t = (\text{Return}_{\text{long},t} - \text{Return}_{\text{short},t}) \times \text{Capital}$$

Insights

- The hedge removes most beta exposure, making returns less sensitive to broad market movements.
- Residual returns primarily reflect the GARCH model's ability to capture idiosyncratic movements.
- Both cases show reduced volatility and smoother equity curves compared to unhedged positions.
- Extreme stock-specific events in the hedge leg can still introduce residual noise.

6.1 Final Results



Figure 11: Final Consolidated Results for All Models and Hedging Strategy

6.2 Final Metrics

For Stock 1

1. Benchmark Return: -8.62
2. Gross Profit: 6.65
3. Max Holding Time: 38
4. Average Holding Time: 9.357142857142858
5. Total Trades: 28
6. Winning Trades: 11
7. Losing Trades: 17
8. Max Drawdown: 6.55
9. Sharpe Ratio (trade-wise): 1.0676
10. Sharpe Ratio (daily): 0.3493

For Stock 2

1. Benchmark Return: -1.15
2. Gross Profit: 22.06
3. Max Holding Time: 47

4. Average Holding Time: 12.375
5. Total Trades: 24
6. Winning Trades: 12
7. Losing Trades: 12
8. Max Drawdown: 7.40
9. Sharpe Ratio (trade-wise): 3.0357
10. Sharpe Ratio (daily): 0.7669